```
module pkt_mux(input [63:0] pkt, [63:0] msg_len, input pad_pkt, zero_p
msglen_pkt, output reg [63:0] o);
always @(*) begin
    if(pad_pkt) o = {1'b1, 63'b 0};
    else if(zero_pkt) o = 64'b0;
    else if(msglen_len) o = msg_len;
    else o = pkt;
end
endmodule
```

```
module dec #(parameter n = 3)
(input [n-1:0] s, input e, output
reg [2**n-1:0] o);
always @(*) begin
    o = (2**n{1'b0});
    if(e) o = 2**s;
end
endmodule
```

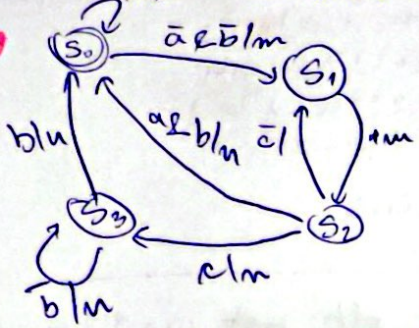
```
module rgst #(parameter n=64)(input ld, clr, clk, rst, input [n-1:0] d
output [n-1:0] q);
always @(posedge clk, negedge rst) begin
    if(!rst) q <= {n{1'b0}};
    else if(clr) q <= {n{1'b0}};
    else if(ld) q <= d;
end
endmodule
module reg_file(input [63:0] d, input [2:0] s, input we, clk, rst, output
[511:0] q);
wire [7:0] w;
dec #(.n(3)) dec3 (.e(we), .s(s), .o(w));
generate; genvar cut; for(cut=0; cut<8; cut=cut+1) begin : rect
    rgst #(.n(64)) rgst64(.ld(w), .clk(clk), .rst(rst), .clr(0), .d(d
.q(q[(cut*64)+63:(cut*64)]));
end
endgenerate
endmodule
```

```
module rgst(input d, clk, rst, set_b, output q);
always @(posedge clk, negedge rst) begin
if(!rst) q<=0; else if(set_b) q<=0; else q<=d;
end
module LFSR(input clk, input rst_b, output [3:0] q);
generate
genvar i;
for(i=0; i<4; i=i+1) begin : rect
    if(i==0) rgst rgst0(.d(q[3]), .q(q[0]), .clk(clk), .set_b(rst_b),
  else if(i==1) rgst rgst1(.d(q[3] ^ q[0]) ...
    else rgst rgstn(.d(q[i-1]), .q(q[i]), .clk(clk), .set_b
(rst_b), .rst(1));
    end generate
    endmodule
```

Scanned with CamScanner

Ex1 AND, NAND → negăm de două ori f
OR, NOR → ~a & ~b = ~(a|b)
→ a & b = ~(~a | ~b)
XOR, XNOR → ~a & b | a & ~b = a ^ b
ā/m,n ~a & ~b | a & b = a ^ b

Ex2



Tabel stări

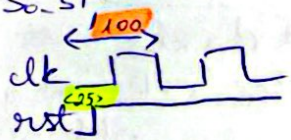
a	b	c	S_0	S_1	S_2	S_3
0	0	0	S_0	S_2	S_1	S_3
0	0	1	S_0	S_2	S_3	S_3
0	1	0	S_0	S_2	S_1	S_0
0	1	1	S_0	:	S_3	S_3
1	0	0	S_1	:	S_1	S_3
1	0	1	S_1	:	S_3	S_3
1	1	0	S_1	:	S_1	S_0
1	1	1	S_1	S_2	S_3	S_0

Tabel ieșiri

a	b	c	S_0	S_1	S_2	S_3
			m n	m n	m n	m n
0	0	0				
0	0	1				
0	1	0				
0	1	1				
1	0	0				
1	0	1				
1	1	0				
1	1	1				

```
module mealy ( input a,b,c,clk,rst,
               output reg m,n);

localparam S0_ST = 2'd0;
localparam S3_ST = 2'd3;
reg [1:0] st_cur, st_next;
always @(*) begin
  case(st_cur)
    S0_ST: if(!a) st_next = S0_ST;
           else if(!b) st_next = S1_ST;
           else st_next = S2_ST;
    S1_ST: st_next = S3_ST;
    S2_ST: if(!c) st_next = S1_ST;
           else st_next = S3_ST;
    S3_ST: if(!b) st_next = S3_ST;
           else st_next = S0_ST;
    default: st_next = S0_ST;
  endcase
end
always @(*) begin
    m=1'b0;
    n=1'b0;
    case(st_cur)
      S0_ST: if(!a) {m,n} = 2'b11;
             else if(!b) n = 1'b1;
             else n = 1'b1;
      S1_ST: m = 1'b1;
      S2_ST: if(c) n = 1'b1;
      S3_ST: n = 1'b1;
    endcase
end
```




```
always @(posedge clk, negedge rst)
if(!rst) begin
    st_cur <= S0_ST;
    {m,n} <= 2'd0;
else
    st_cur <= st_next;
end
endmodule
module mealy_tb;
 reg a,b,c,clk,rst;
 wire m,n;
mealy uut(.a(a),.b(b),.c(c),
 .clk(clk),.rst(rst),.m(m),
 .n(n));
 initial begin
    clk=0;
    forever #50 clk = ~clk;
end
initial begin
  $monitor("Timp=%t | Rst=%b |
a=%b | b=%b | c=%b | m=%b |
n=%b ", $time,rst,a,b,c,
 m,n);
 a=0; b=0; c=0;
#100 { rst=0; #25 rst=1;
  #75 mai a,b,c...
    $finish
end
endmodule
```